

Metw Laravel Backend Flow Documentation After AI Prompts

بالكامل، **flow** يبقى عندك مستند واحد يشرح ال prompts، يطبق كل ال AI Agent الهدف من الملف ده: بعد ما ال الجديدة تتركب على المشروع الحالي بدون كسر modules بتخدم إيه، وإزاي ال table كل جزء مكانه فين في الكود، كل الموجود.

1. Current Project Reality قبل التعديل

وفيه بالفعل أجزاء مهمة جاهزة Laravel المشروع الحالي

Existing core paths

```
routes/api.php
routes/web.php
app/Models/
app/Http/Controllers/Api/V1/
app/Http/Controllers/Dashboard/
app/Http/Requests/
app/Http/Resources/
app/Services/
database/migrations/
database/seeder/
resources/views/dashboard/
```

Existing important models

```
app/Models/User.php
app/Models/Vendor.php
app/Models/Warehouse.php
app/Models/ShipmentCompany.php
app/Models/UserAddress.php
app/Models/VendorBranch.php
app/Models/PackageAddress.php
app/Models/Package.php
app/Models/PackageDetails.php
app/Models/PackageImage.php
app/Models/Order.php
app/Models/OrderItem.php
app/Models/Product.php
app/Models/ProductMedia.php
app/Models/ProductVariant.php
app/Models/EcommerceOrder.php
app/Models/EcommerceOrderItem.php
```

```
app/Models/Governorate.php
app/Models/City.php
```

Existing API areas

```
app/Http/Controllers/Api/V1/AuthController.php
app/Http/Controllers/Api/V1/UserController.php
app/Http/Controllers/Api/V1/Ecommerce/
app/Http/Controllers/Api/V1/Shipment/
```

Existing dashboard areas

```
app/Http/Controllers/Dashboard/Admin/
app/Http/Controllers/Dashboard/Vendor/
app/Http/Controllers/Dashboard/ShipmentCompany/
```

Current issue

كثير بتكرر نفس الفكرة tables المشروع فيه:

```
user_addresses
vendor_branches
package_addresses
warehouses address columns
shipment_companies address/location columns
```

ده مع الوقت بيعمل مشاكل

- أي تغيير في شكل العنوان لازم يتكرر في كذا مكان.
- validation بتتكرر.
- شكلهم مختلف response بيحتاج يتعامل مع كذا frontend.
- duplication جديد بيزود migration أي.

تدريجي migration جديدة ونخلي القديم شغال لحد ما نعمل **shared structures** هو إننا نضيف senior backend الحل

2. Business Requirement من ال PDFs

المستندات بتطلب 4 أنظمة رئيسية:

1. **MetwLogistic**: website للخدمات اللوجستية.
2. **Metwzon**: ecommerce marketplace.
3. **MetwExpress**: شحن وتوصيل سريع.
4. **Metwgo**: تطبيق المناديب.

تاريخ فتح الحساب، auto، بتطلب بيانات مشتركة لفتح الحسابات مثل الاسم الأول، اسم الأب، اللقب، رقم الحساب PDFs ال
password flow، تاريخ ميلاد، النوع، email verification، رقمين موبايل، auto،

address format كمان بتطلب governorate, city filtered by governorate, district/village name, area type, street, side street, building number, floor number, building name, landmark, description.

كامل للمناديب فيه module وتطلب:

- نوع المندوب: حر أو تابع لمستودع.
- نوع العمل: توصيل داخل المحافظة، شحن بين المحافظات، سائق باص.
- محافظات ومدن الخدمة.
- وسيلة النقل.
- مستندات وصور.
- حدود وزن وحجم حسب وسيلة النقل.

3. Final Target Architecture

بالشكل ده architecture المفروض يبقى عندك، prompts بعد تنفيذ ال

```
Auth Layer
├─ users
├─ vendors
├─ warehouses / shipment_companies legacy
├─ admins / employees

Shared Profile Layer
├─ account_profiles
├─ addresses
├─ media_files

Representative Layer
├─ representatives
├─ representative_work_types
├─ representative_service_governorates
├─ representative_service_cities
├─ representative_vehicles
├─ transport_types

Shipment Layer
├─ shipment_contacts
├─ shipment_requests
├─ shipment_request_packages
├─ media_files package images

Vendor/Warehouse Completion Layer
├─ vendor_business_profiles
├─ warehouse_business_profiles
├─ media_files documents

Product Enhancement Layer
├─ product_shipping_profiles
```

```
|— product_return_policies
|— product_shipping_fees
|— existing_products/product_media/product_variants
```

4. New Code Location Map

جدید لازم یکون مکانه واضح بالشکل ده module کل

A. Account Profile Flow

Business purpose

users, vendors, warehouses. کثیر فی columns ده هیكون مکان البیانات المشتركة لكل أنواع الحسابات بدل ما نزود

Database

```
database/migrations/xxxx_xx_xx_XXXXXX_create_account_profiles_table.php
```

Model

```
app/Models/AccountProfile.php
```

Service

```
app/Services/Account/CompleteAccountProfileService.php
```

Requests

```
app/Http/Requests/Account/CompleteAccountProfileRequest.php
app/Http/Requests/Account/UpdateAccountProfileRequest.php
```

Resources

```
app/Http/Resources/Account/AccountProfileResource.php
```

Controller

```
app/Http/Controllers/Api/V1/Account/AccountProfileController.php
```

Routes

Add in:

```
routes/api.php
```

Expected routes:

```
Route::prefix('v1/account')->middleware('auth:sanctum')->group(function () {  
    Route::get('profile', [AccountProfileController::class, 'show']);  
    Route::post('profile', [AccountProfileController::class, 'store']);  
    Route::put('profile', [AccountProfileController::class, 'update']);  
});
```

Main flow

```
User logs in  
→ frontend checks /api/v1/account/profile  
→ if no profile or status incomplete  
→ user completes common account data  
→ service validates and saves account_profiles  
→ status becomes pending_review or approved depending on account type
```

Table responsibility

```
account_profiles
```

Stores:

- first_name
- father_name
- last_name
- account_number
- account_opening_date
- primary_mobile
- secondary_mobile
- mobile verification timestamps
- birth_date

- gender
- national_id optional
- status

Relationships

In `AccountProfile.php`:

```
public function profileable()
{
    return $this->morphTo();
}
```

In `User.php`, `Vendor.php`, `Warehouse.php`:

```
public function accountProfile()
{
    return $this->morphOne(AccountProfile::class, 'profileable');
}
```

B. Shared Address Flow

Business purpose

كل العناوين في النظام تبقى بنفس الشكل، بدل التكرار في `user_addresses`, `vendor_branches`, `package_addresses`.

Database

```
database/migrations/xxxx_xx_xx_XXXXXX_create_addresses_table.php
```

Model

```
app/Models/Address.php
```

Service

```
app/Services/Address/AddressService.php
```

Requests

```
app/Http/Requests/Address/StoreAddressRequest.php  
app/Http/Requests/Address/UpdateAddressRequest.php
```

Resource

```
app/Http/Resources/Address/AddressResource.php
```

Optional Controller

```
app/Http/Controllers/Api/V1/Address/AddressController.php
```

Existing old code that will be gradually replaced

```
app/Models/UserAddress.php  
app/Models/VendorBranch.php  
app/Models/PackageAddress.php  
app/Http/Controllers/Api/V1/Ecommerce/User/UserAddressController.php  
app/Http/Controllers/Api/V1/Shipment/Order/PackageAddressController.php  
app/Http/Controllers/Dashboard/Vendor/VendorBranchController.php
```

Main flow

```
Any module needs address  
→ calls AddressService  
→ AddressService validates governorate_id/city_id  
→ checks city belongs to governorate  
→ saves to addresses with addressable_type/addressable_id
```

Address types

```
user_delivery  
vendor_branch  
warehouse_location  
shipment_sender  
shipment_receiver  
representative_home
```

Important rule

Do not delete old address tables now. Use **addresses** for all new modules, then migrate old data later.

C. Media Files Flow

Business purpose

واحدة لكل الملفات table كثير لكل صورة أو مستند، نستخدم columns بدل ما نعمل

Database

```
database/migrations/xxxx_xx_xx_xxxxxx_create_media_files_table.php
```

Model

```
app/Models/MediaFile.php
```

Service

```
app/Services/Media/UploadMediaService.php
```

Resource

```
app/Http/Resources/Media/MediaFileResource.php
```

Main flow

```
Frontend uploads file
→ Controller receives file
→ UploadMediaService stores file on disk
→ creates media_files row
→ owner_type/owner_id links file to representative/vendor/product/package/etc.
```

Collections


```
profile_photo
national_id_front
national_id_back
vehicle_photo
driver_license_front
driver_license_back
vehicle_license_front
vehicle_license_back
commercial_register_front
commercial_register_back
tax_card_front
tax_card_back
chamber_commerce
store_sign
store_logo
warehouse_inside
warehouse_outside
product_image
product_video
package_image
```

Existing old code related to media

```
app/Models/ProductMedia.php
app/Models/PackageImage.php
```

Keep them working. New modules use `media_files`.

D. Transport Types Flow

Business purpose

hardcoded، ثابتة، مش table تتخزن في PDF وسائل النقل ال 16 اللي في

Database

```
database/migrations/xxxx_xx_xx_XXXXXX_create_transport_types_table.php
```

Model

```
app/Models/TransportType.php
```

Seeder

```
database/seeder/TransportTypeSeeder.php
```

Register in:

```
database/seeder/DatabaseSeeder.php
```

Controller

```
app/Http/Controllers/Api/V1/Representative/TransportTypeController.php
```

Route

```
Route::get('/api/v1/transport-types', [TransportTypeController::class, 'index']);
```

or under representatives:

```
Route::get('/api/v1/representatives/transport-types',  
[TransportTypeController::class, 'index']);
```

Main flow

```
Frontend opens representative registration  
→ calls transport-types endpoint  
→ user selects transport type  
→ representative_vehicle stores selected transport_type_id  
→ system can later calculate max allowed weight/volume
```

E. Representative / Courier Flow

Business purpose

Metwgo. تطبيق المناديب module ده

Database

```
create_representatives_table.php
create_representative_work_types_table.php
create_representative_service_governorates_table.php
create_representative_service_cities_table.php
create_representative_vehicles_table.php
```

Models

```
app/Models/Representative.php
app/Models/RepresentativeWorkType.php
app/Models/RepresentativeServiceGovernorate.php
app/Models/RepresentativeServiceCity.php
app/Models/RepresentativeVehicle.php
```

Services

```
app/Services/Representative/RegisterRepresentativeService.php
app/Services/Representative/UpdateRepresentativeService.php
app/Services/Representative/UploadRepresentativeDocumentService.php
app/Services/Representative/ApproveRepresentativeService.php
app/Services/Representative/RejectRepresentativeService.php
```

Requests

```
app/Http/Requests/Representative/RegisterRepresentativeRequest.php
app/Http/Requests/Representative/UpdateRepresentativeRequest.php
app/Http/Requests/Representative/UploadRepresentativeDocumentRequest.php
app/Http/Requests/Representative/ApproveRepresentativeRequest.php
app/Http/Requests/Representative/RejectRepresentativeRequest.php
```

Resources

```
app/Http/Resources/Representative/RepresentativeResource.php
app/Http/Resources/Representative/RepresentativeVehicleResource.php
app/Http/Resources/Representative/TransportTypeResource.php
```

API Controllers

```
app/Http/Controllers/Api/V1/Representative/RepresentativeController.php
app/Http/Controllers/Api/V1/Representative/RepresentativeDocumentController.php
```

```
app/Http/Controllers/Api/V1/Representative/TransportTypeController.php
```

Admin Dashboard Controllers

```
app/Http/Controllers/Dashboard/Admin/RepresentativeController.php
```

Views if dashboard web is needed

```
resources/views/dashboard/admin/representatives/index.blade.php  
resources/views/dashboard/admin/representatives/show.blade.php
```

Routes in `routes/api.php`

```
Route::prefix('v1/representatives')->middleware('auth:sanctum')->group(function ()  
{  
    Route::get('transport-types', [TransportTypeController::class, 'index'])->  
>withoutMiddleware('auth:sanctum');  
    Route::post('register', [RepresentativeController::class, 'register']);  
    Route::get('me', [RepresentativeController::class, 'me']);  
    Route::put('me', [RepresentativeController::class, 'update']);  
    Route::post('documents', [RepresentativeDocumentController::class, 'store']);  
});
```

Routes in `routes/web.php`

Inside admin dashboard group:

```
Route::prefix('representatives')->name('representatives.')->group(function () {  
    Route::get('/', [RepresentativeController::class, 'index'])->name('index');  
    Route::get('/{representative}', [RepresentativeController::class, 'show'])->  
>name('show');  
    Route::post('/{representative}/approve', [RepresentativeController::class,  
'approve'])->name('approve');  
    Route::post('/{representative}/reject', [RepresentativeController::class,  
'reject'])->name('reject');  
});
```

Main registration flow

1. User registers/login normally through existing AuthController.
2. User opens representative registration.

3. Frontend loads transport types.
4. User selects account_type:
 - free
 - warehouse
5. If account_type = warehouse, warehouse_id is required.
6. User selects work types.
7. User selects service governorates.
8. If local_delivery, user selects cities inside selected governorate.
9. User adds vehicle data.
10. User uploads required documents.
11. representative status becomes pending_review.
12. Admin reviews and approves/rejects.
13. If approved, representative can receive assignments later.

Important validation rules

```
account_type = warehouse requires warehouse_id
local_delivery allows one governorate only
local_delivery can select multiple cities from same governorate
inter_governorate_shipping can select multiple governorates
bus_driver can select multiple governorates
invalid work type combinations must be rejected
transport_type_id must exist and be active
```

F. Shipment Contacts Flow

Business purpose

بنتقول إن مستخدم تطبيق الشحن عنده قائمتين PDF الـ

```
senders list
receivers list
```

له اسم ورقم موبايل وعنوان محفوظ sender/receiver كل

Database

```
create_shipment_contacts_table.php
```

Address uses:

```
addresses
```

Model

```
app/Models/ShipmentContact.php
```

Service

```
app/Services/Shipment/CreateShipmentContactService.php  
app/Services/Shipment/UpdateShipmentContactService.php
```

Requests

```
app/Http/Requests/Shipment/StoreShipmentContactRequest.php  
app/Http/Requests/Shipment/UpdateShipmentContactRequest.php
```

Resource

```
app/Http/Resources/Shipment/ShipmentContactResource.php
```

Controller

```
app/Http/Controllers/Api/V1/Shipment/ShipmentContactController.php
```

Routes

```
Route::prefix('v1/shipment/contacts')->middleware('auth:sanctum')->group(function  
( ) {  
    Route::get('/', [ShipmentContactController::class, 'index']);  
    Route::post('/', [ShipmentContactController::class, 'store']);  
    Route::get('/{contact}', [ShipmentContactController::class, 'show']);  
    Route::put('/{contact}', [ShipmentContactController::class, 'update']);  
    Route::delete('/{contact}', [ShipmentContactController::class, 'destroy']);  
});
```

Main flow

```
User opens shipment form
→ user needs sender and receiver
→ frontend calls shipment contacts
→ if no sender/receiver, user creates them
→ each contact saves address through AddressService
→ shipment request later references sender_contact_id and receiver_contact_id
```

G. Shipment Request Flow

Business purpose

هذه flow طلب الشحن الجديد بشكل clean: shipment route + multiple packages.

Database

```
create_shipment_requests_table.php
create_shipment_request_packages_table.php
```

Models

```
app/Models/ShipmentRequest.php
app/Models/ShipmentRequestPackage.php
```

Services

```
app/Services/Shipment/CreateShipmentRequestService.php
app/Services/Shipment/AddShipmentPackageService.php
app/Services/Shipment/SubmitShipmentRequestService.php
```

Requests

```
app/Http/Requests/Shipment/StoreShipmentRequestRequest.php
app/Http/Requests/Shipment/StoreShipmentRequestPackageRequest.php
app/Http/Requests/Shipment/SubmitShipmentRequestRequest.php
```

Resources

```
app/Http/Resources/Shipment/ShipmentRequestResource.php
app/Http/Resources/Shipment/ShipmentRequestPackageResource.php
```

Controller

```
app/Http/Controllers/Api/V1/Shipment/ShipmentRequestController.php
app/Http/Controllers/Api/V1/Shipment/ShipmentRequestPackageController.php
```

Existing shipment code that should remain working

```
app/Models/Package.php
app/Models/PackageDetails.php
app/Models/PackageAddress.php
app/Models/PackageImage.php
app/Models/Order.php
app/Models/OrderItem.php
app/Http/Controllers/Api/V1/Shipment/Order/
app/Services/PackageService.php
app/Services/ShippingService.php
```

Routes

```
Route::prefix('v1/shipment/requests')->middleware('auth:sanctum')->group(function
() {
    Route::get('/', [ShipmentRequestController::class, 'index']);
    Route::post('/', [ShipmentRequestController::class, 'store']);
    Route::get('/{shipmentRequest}', [ShipmentRequestController::class, 'show']);
    Route::post('/{shipmentRequest}/packages',
[ShipmentRequestPackageController::class, 'store']);
    Route::delete('/{shipmentRequest}/packages/{package}',
[ShipmentRequestPackageController::class, 'destroy']);
    Route::post('/{shipmentRequest}/submit', [ShipmentRequestController::class,
'submit']);
});
```

Main flow

1. User creates sender contact.
2. User creates receiver contact.
3. User creates shipment_request with sender_contact_id and receiver_contact_id.
4. System creates shipment_number.
5. User adds one or more shipment_request_packages.

6. Each package has dimensions, weight, quantity, package type, storage conditions.
7. User uploads package images to media_files.
8. User submits shipment_request.
9. status moves from draft to pending_pricing or confirmed depending current business rule.

Why not remove old Package/Order now?

Because existing APIs and checkout logic depend on:

```
CartController  
CheckoutController  
OrderController  
PackageService  
ShippingService
```

So the new flow should be added safely, then old flow can be migrated later.

H. Vendor Business Profile Completion Flow

Business purpose

JI vendor approval: national ID, commercial register, tax card, chamber commerce, store logo/sign, branches. لكن PDF الحالي موجود، seller بيانات قبل approval: national ID, commercial register, tax card, chamber commerce, store logo/sign, branches.

Database

```
create_vendor_business_profiles_table.php
```

Branches can still use current:

```
vendor_branches
```

or new:

```
addresses where address_type = vendor_branch
```

Model

```
app/Models/VendorBusinessProfile.php
```

Service

```
app/Services/Vendor/CompleteVendorBusinessProfileService.php  
app/Services/Vendor/ApproveVendorBusinessProfileService.php  
app/Services/Vendor/RejectVendorBusinessProfileService.php
```

Requests

```
app/Http/Requests/Vendor/CompleteVendorBusinessProfileRequest.php  
app/Http/Requests/Vendor/ApproveVendorBusinessProfileRequest.php  
app/Http/Requests/Vendor/RejectVendorBusinessProfileRequest.php
```

Resource

```
app/Http/Resources/Vendor/VendorBusinessProfileResource.php
```

API Controller

```
app/Http/Controllers/Api/V1/Ecommerce/VendorBusinessProfileController.php
```

Dashboard Controller

```
app/Http/Controllers/Dashboard/Admin/VendorBusinessProfileController.php
```

or add methods into existing:

```
app/Http/Controllers/Dashboard/Admin/VendorController.php
```

Existing vendor code

```
app/Models/Vendor.php  
app/Models/VendorBranch.php  
app/Http/Controllers/Api/V1/Ecommerce/VendorController.php
```

```
app/Http/Controllers/Dashboard/Vendor/ProfileController.php  
app/Http/Controllers/Dashboard/Vendor/VendorBranchController.php  
app/Http/Controllers/Dashboard/Admin/VendorController.php
```

Main flow

1. Vendor registers with existing vendor registration.
2. Vendor logs in.
3. System checks vendor_business_profiles.
4. If missing/incomplete, vendor must complete profile.
5. Vendor uploads documents through media_files.
6. Vendor adds branch shipping address.
7. status becomes pending_review.
8. Admin approves/rejects.
9. Vendor can add products only if approved.

I. Warehouse Business Profile Completion Flow

Business purpose

PDF says rename concept from shipment company account to warehouse account, but code currently has `ShipmentCompany` and `Warehouse`. Do not rename everything at once.

Safe approach

Keep legacy code:

```
app/Models/ShipmentCompany.php  
app/Http/Controllers/Dashboard/ShipmentCompany/  
app/Http/Controllers/Api/V1/Shipment/ShipmentCompanyController.php
```

Add warehouse profile completion beside it:

```
warehouse_business_profiles
```

Database

```
create_warehouse_business_profiles_table.php
```

Model

```
app/Models/WarehouseBusinessProfile.php
```

Service

```
app/Services/Warehouse/CompleteWarehouseBusinessProfileService.php  
app/Services/Warehouse/ApproveWarehouseBusinessProfileService.php  
app/Services/Warehouse/RejectWarehouseBusinessProfileService.php
```

Requests

```
app/Http/Requests/Warehouse/CompleteWarehouseBusinessProfileRequest.php  
app/Http/Requests/Warehouse/ApproveWarehouseBusinessProfileRequest.php  
app/Http/Requests/Warehouse/RejectWarehouseBusinessProfileRequest.php
```

Controllers

```
app/Http/Controllers/Api/V1/Warehouse/WarehouseBusinessProfileController.php  
app/Http/Controllers/Dashboard/Admin/WarehouseBusinessProfileController.php
```

Main flow

1. Warehouse account is created/login.
2. Warehouse completes owner national ID and establishment data.
3. Warehouse uploads documents to media_files.
4. Warehouse adds location/address using addresses.
5. status becomes pending_review.
6. Admin approves/rejects.

Important warning

Do not globally rename `shipment_company_id` yet. It exists in many tables and services. Rename later only after a full migration plan.

J. Product Cleanup Flow

Business purpose

Products table already has many columns. New shipping/return/business logic should move into separated tables.

Existing product code

```
app/Models/Product.php
app/Models/ProductMedia.php
app/Models/ProductVariant.php
app/Models/ProductColor.php
app/Models/ProductSize.php
app/Http/Controllers/Api/V1/Ecommerce/Product/ProductController.php
app/Http/Controllers/Dashboard/Vendor/ProductController.php
app/Http/Controllers/Dashboard/Admin/ProductController.php
app/Http/Requests/StoreProductRequest.php
app/Http/Requests/UpdateProductRequest.php
```

New database

```
create_product_shipping_profiles_table.php
create_product_return_policies_table.php
create_product_shipping_fees_table.php
```

New models

```
app/Models/ProductShippingProfile.php
app/Models/ProductReturnPolicy.php
app/Models/ProductShippingFee.php
```

Services

```
app/Services/Product/CreateProductService.php
app/Services/Product/UpdateProductService.php
app/Services/Product/ProductShippingProfileService.php
app/Services/Product/ProductReturnPolicyService.php
```

If the current product logic is still inside controller, move it gradually into services.

Main flow

1. Vendor creates product.
2. Product basic data saves in products.
3. Product images/videos save in ProductMedia or media_files depending migration

stage.

4. Stock/price/discount remain compatible with current products table.
5. Shipping dimensions and package type save in product_shipping_profiles.
6. Return rules save in product_return_policies.
7. Shipping fee rules save in product_shipping_fees.
8. API resource returns product with nested shipping_profile, return_policy, shipping_fee.

5. Database Table Responsibility Summary

Table	Responsibility	New/Old
users	App users login/auth	Existing
vendors	Seller login/basic data	Existing
warehouses	Warehouse basic entity	Existing
shipment_companies	Legacy shipping company logic	Existing legacy
account_profiles	Shared account completion data	New
addresses	Shared polymorphic addresses	New
media_files	Shared polymorphic uploads/documents	New
transport_types	16 courier transport types	New
representatives	Courier main account profile	New
representative_work_types	Courier selected work modes	New
representative_service_governorates	Courier governorate coverage	New
representative_service_cities	Courier city coverage	New
representative_vehicles	Courier vehicle data	New
shipment_contacts	Saved sender/receiver contacts	New
shipment_requests	New shipment request route	New
shipment_request_packages	Packages inside shipment request	New
vendor_business_profiles	Seller approval/business data	New
warehouse_business_profiles	Warehouse approval/business data	New
product_shipping_profiles	Product package/shipping details	New
product_return_policies	Product return rules	New
product_shipping_fees	Product shipping fee rules	New

6. API Flow Map for Frontend

6.1 Account completion

```
GET    /api/v1/account/profile
POST   /api/v1/account/profile
PUT    /api/v1/account/profile
```

Frontend usage:

```
After login
→ check profile
→ if incomplete, redirect user to complete profile screen
```

6.2 Representative app

```
GET    /api/v1/representatives/transport-types
POST   /api/v1/representatives/register
GET    /api/v1/representatives/me
PUT    /api/v1/representatives/me
POST   /api/v1/representatives/documents
```

Frontend usage:

```
Metwgo registration screen
→ get transport types
→ send representative registration
→ upload documents
→ wait for approval
```

6.3 Shipment contacts

```
GET    /api/v1/shipment/contacts?type=sender
GET    /api/v1/shipment/contacts?type=receiver
POST   /api/v1/shipment/contacts
GET    /api/v1/shipment/contacts/{id}
PUT    /api/v1/shipment/contacts/{id}
DELETE /api/v1/shipment/contacts/{id}
```

Frontend usage:

Shipment request screen

- user selects sender from saved list
- user selects receiver from saved list
- if missing, create new sender/receiver

6.4 Shipment request

```
GET    /api/v1/shipment/requests
POST   /api/v1/shipment/requests
GET    /api/v1/shipment/requests/{id}
POST   /api/v1/shipment/requests/{id}/packages
DELETE /api/v1/shipment/requests/{id}/packages/{packageId}
POST   /api/v1/shipment/requests/{id}/submit
```

Frontend usage:

- User chooses sender/receiver
- creates shipment request draft
 - adds packages
 - uploads package images
 - submits shipment

6.5 Vendor completion

```
GET    /api/v1/ecommerce/vendor/business-profile
POST   /api/v1/ecommerce/vendor/business-profile
PUT    /api/v1/ecommerce/vendor/business-profile
POST   /api/v1/ecommerce/vendor/business-profile/documents
```

Frontend usage:

- Vendor dashboard
- if profile not approved, block product publishing
 - vendor completes business profile
 - admin approves

6.6 Warehouse completion


```
GET    /api/v1/warehouse/business-profile
POST   /api/v1/warehouse/business-profile
PUT    /api/v1/warehouse/business-profile
POST   /api/v1/warehouse/business-profile/documents
```

7. Admin Dashboard Flow

New admin modules

```
Representatives
Vendor Business Profiles
Warehouse Business Profiles
Transport Types optional management
```

Controller paths

```
app/Http/Controllers/Dashboard/Admin/RepresentativeController.php
app/Http/Controllers/Dashboard/Admin/VendorBusinessProfileController.php
app/Http/Controllers/Dashboard/Admin/WarehouseBusinessProfileController.php
```

View paths

```
resources/views/dashboard/admin/representatives/index.blade.php
resources/views/dashboard/admin/representatives/show.blade.php
resources/views/dashboard/admin/vendor-business-profiles/index.blade.php
resources/views/dashboard/admin/vendor-business-profiles/show.blade.php
resources/views/dashboard/admin/warehouse-business-profiles/index.blade.php
resources/views/dashboard/admin/warehouse-business-profiles/show.blade.php
```

Admin approval status flow

```
incomplete
→ pending_review
→ approved
```

or:

```
incomplete
→ pending_review
```

- rejected
- pending_review after user updates
- approved

8. Validation Rules Summary

Address

```
governorate_id required exists:governorates,id
city_id required exists:cities,id
city must belong to governorate
area_type required in:district,village
building_number nullable integer
floor_number nullable integer
```

Account profile

```
first_name required string
father_name required string
last_name required string
primary_mobile required unique
secondary_mobile nullable different:primary_mobile
birth_date required date before:today
gender required in:male,female
```

Representative

```
account_type required in:free,warehouse
warehouse_id required_if:account_type,warehouse
work_types required array
transport_type_id required exists:transport_types,id
service_governorates required array
service_cities required if work_type includes local_delivery
```

Shipment contact

```
type required in:sender,receiver
full_name required string
primary_mobile required
address required array
```

Shipment request

```
sender_contact_id required exists shipment_contacts where type sender and user_id = auth user
receiver_contact_id required exists shipment_contacts where type receiver and user_id = auth user
packages required before submit
```

Product

```
product required fields remain in StoreProductRequest
shipping profile optional/required based on business decision
return policy required if product is sellable
shipping fee policy required if product needs shipping
```

9. Testing Map

Create tests under:

```
tests/Feature/Account/
tests/Feature/Address/
tests/Feature/Representative/
tests/Feature/Shipment/
tests/Feature/Vendor/
tests/Feature/Warehouse/
tests/Feature/Product/
```

Suggested test files:

```
tests/Feature/Account/CompleteAccountProfileTest.php
tests/Feature/Address/AddressValidationTest.php
tests/Feature/Representative/RegisterRepresentativeTest.php
tests/Feature/Representative/RepresentativeDocumentUploadTest.php
tests/Feature/Shipment/ShipmentContactTest.php
tests/Feature/Shipment/ShipmentRequestTest.php
tests/Feature/Vendor/VendorBusinessProfileTest.php
tests/Feature/Warehouse/WarehouseBusinessProfileTest.php
tests/Feature/Product/ProductShippingAndReturnPolicyTest.php
```

Run:

```
php artisan test
```

or:

```
composer test
```

10. Migration Order

Use this order to avoid broken foreign keys:

1. create_account_profiles_table
2. create_addresses_table
3. create_media_files_table
4. create_transport_types_table
5. create_representatives_table
6. create_representative_work_types_table
7. create_representative_service_governorates_table
8. create_representative_service_cities_table
9. create_representative_vehicles_table
10. create_shipment_contacts_table
11. create_shipment_requests_table
12. create_shipment_request_packages_table
13. create_vendor_business_profiles_table
14. create_warehouse_business_profiles_table
15. create_product_shipping_profiles_table
16. create_product_return_policies_table
17. create_product_shipping_fees_table

11. Important Do Not Break Rules

Do not delete users table.
Do not delete vendors table.
Do not delete shipment_companies table.
Do not rename shipment_company_id in existing tables yet.
Do not delete user_addresses/vendor_branches/package_addresses yet.
Do not move all product columns at once.
Do not put business logic inside controllers.
Do not store file paths directly as many columns when media_files can handle it.
Do not skip tests.

12. What should happen after AI finishes all prompts

The final project should have:

```
New migrations created
New models created
New service classes created
New Form Requests created
New API Resources created
New API controllers created
New admin approval controllers/views if needed
Transport types seeded
Routes added safely
Tests added
Docs created
Existing flows still working
```

Then run:

```
php artisan migrate
php artisan db:seed --class=TransportTypeSeeder
php artisan test
```

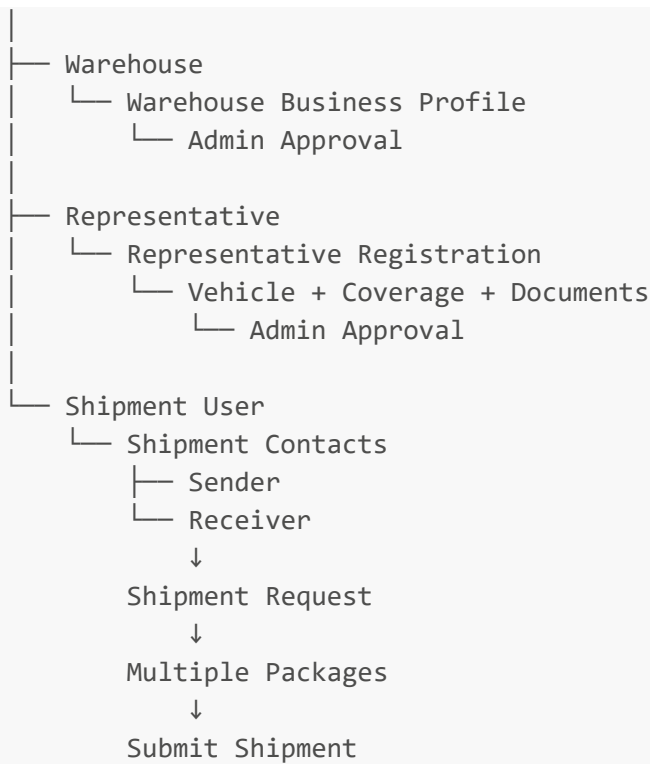
For local full reset only if safe:

```
php artisan migrate:fresh --seed
php artisan test
```

Do not run `migrate:fresh` on production.

13. Final Backend Flow Diagram

```
Login/Register
↓
Account Profile Completion
↓
Choose module
├── Ecommerce User
│   ├── User Delivery Addresses
│   └── Product Orders
├── Vendor
│   ├── Vendor Business Profile
│   │   ├── Admin Approval
│   │   └── Vendor Branches
│   │       ├── Product Create
│   │       └── Product Shipping/Return Rules
```



14. Clean Coding Standard for AI Agent

Any generated code should follow this structure:

```

Controller = receive request only
FormRequest = validation only
Service = business logic
Model = relationships and casts
Resource = API response shape
Seeder = fixed lookup data
Test = behavior verification
  
```

Bad pattern:

```

public function store(Request $request)
{
    // 200 lines validation + upload + DB logic + response
}
  
```

Good pattern:

```

public function store(StoreShipmentContactRequest $request)
{
    $contact = $this->service->create($request->user(), $request->validated());
}
  
```

```
    return new ShipmentContactResource($contact);  
}
```

15. Final Checklist

Before saying the work is complete, verify:

- [] All migrations run successfully.
- [] Transport types seeded correctly.
- [] Account profile can be created.
- [] AddressService validates city/governorate relation.
- [] Representative registration works.
- [] Representative documents upload works.
- [] Admin can approve/reject representative.
- [] Shipment sender/receiver contacts work.
- [] Shipment request supports multiple packages.
- [] Vendor business profile completion works.
- [] Warehouse business profile completion works.
- [] Product shipping profile saves correctly.
- [] Product return policy saves correctly.
- [] Old shipment/order APIs still work.
- [] Old ecommerce APIs still work.
- [] Tests pass.

16. Recommended Next Step

ينفذ، اطلب منه يعمل AI Agent بعد ما الـ

- Generate a final implementation report showing:
1. Files created
 2. Files modified
 3. New database tables
 4. New API routes
 5. Test results
 6. Any old code that still needs future refactor

ده هيخليك تراجع الشغل بسرعة وتعرف هل كل حاجة اتحطت في مكانها الصح ولا لا.